tf.pad Stay organized with collections Save and categorize content based on your preferences.

https://www.tensorflow.org/api_docs/python/tf/pad

> *Pads a tensor.*

Function Signatures

```
tf.pad( tensor, paddings, mode='CONSTANT', constant_values=0, name=None )
```

Code Examples

```
tf.pad(
tensor, paddings, mode='CONSTANT', constant_values=0, name=None
)
```


```
tf.pad(
tensor, paddings, mode='CONSTANT', constant_values=0, name=None
)
```

```
t = tf.constant([[1, 2, 3], [4, 5, 6]])
paddings = tf.constant([[1, 1,], [2, 2]])
# 'constant_values' is 0.
# rank of 't' is 2.
tf.pad(t, paddings, "CONSTANT")  # [[0, 0, 0, 0, 0, 0, 0],
#  [0, 0, 1, 2, 3, 0, 0],
#  [0, 0, 4, 5, 6, 0, 0],
#  [0, 0, 0, 0, 0, 0, 0]]
tf.pad(t, paddings, "REFLECT")  # [[6, 5, 4, 5, 6, 5, 4],
#  [3, 2, 1, 2, 3, 2, 1],
#  [6, 5, 4, 5, 6, 5, 4],
#  [3, 2, 1, 2, 3, 2, 1]]
tf.pad(t, paddings, "SYMMETRIC")  # [[2, 1, 1, 2, 3, 3, 2],
#  [2, 1, 1, 2, 3, 3, 2],
#  [5, 4, 4, 5, 6, 6, 5],
#  [5, 4, 4, 5, 6, 6, 5]]
```


```
t = tf.constant([[1, 2, 3], [4, 5, 6]])
paddings = tf.constant([[1, 1,], [2, 2]])
# 'constant_values' is 0.
# rank of 't' is 2.
tf.pad(t, paddings, "CONSTANT")  # [[0, 0, 0, 0, 0, 0, 0],
#  [0, 0, 1, 2, 3, 0, 0],
#  [0, 0, 4, 5, 6, 0, 0],
#  [0, 0, 0, 0, 0, 0, 0]]
tf.pad(t, paddings, "REFLECT")  # [[6, 5, 4, 5, 6, 5, 4],
#  [3, 2, 1, 2, 3, 2, 1],
#  [6, 5, 4, 5, 6, 5, 4],
#  [3, 2, 1, 2, 3, 2, 1]]
tf.pad(t, paddings, "SYMMETRIC")  # [[2, 1, 1, 2, 3, 3, 2],
#  [2, 1, 1, 2, 3, 3, 2],
#  [5, 4, 4, 5, 6, 6, 5],
#  [5, 4, 4, 5, 6, 6, 5]]
```


Documentation

Pads a tensor.

Used in the notebooks

• Advanced automatic differentiation

- Client-efficient large-model federated learning via `federated_select` and sparse aggregation
- A Tour of TensorFlow Probability
- Multiple changepoint detection and Bayesian model selection
- Graph-based Neural Structured Learning in TFX

This operation pads a tensor according to the paddings you specify. paddings is an integer tensor with shape [n, 2], where n is the rank of tensor. For each dimension D of input, paddings[D, 0] indicates how many values to add before the contents of tensor in that dimension, and paddings[D, 1] indicates how many values to add after the contents of tensor in that dimension. If mode is "REFLECT" then both paddings[D, 0] and paddings[D, 1] must be no greater than tensor.dim_size(D) - 1. If mode is "SYMMETRIC" then both paddings[D, 0] and paddings[D, 1] must be no greater than tensor.dim_size(D).

The padded size of each dimension D of the output is:

paddings[D, 0] + tensor.dim_size(D) + paddings[D, 1]

For example:

Returns

Raises